

Learning while Deploying: Fleet-Scale Reinforcement Learning for Generalist Robot Policies

Yi Wang^{1,2} Xinchun Li² Pengwei Xie² Pu Yang² Buqing Nie²
Yunuo Cai^{1,2} Qinglin Zhang² Chendi Qu² Jeffrey Wu³ Jianheng Song²
Xinlin Ren² Jingshun Huang^{1,2} Mingjie Pan^{1,2} Siyuan Feng² Zhi Chen² Jianlan Luo^{1,2†}
¹Shanghai Innovation Institute. ²AGIBOT Finch. ³Columbia University.

<https://finch.agibot.com/research/lwd>

Abstract—Generalist robot policies increasingly benefit from large-scale pretraining, but offline data alone is insufficient for robust real-world deployment. Deployed robots encounter distribution shifts, long-tail failures, task variations, and human correction opportunities that fixed demonstration datasets cannot fully capture. We present *Learning While Deploying* (LWD), a fleet-scale offline-to-online reinforcement learning framework for continual post-training of generalist Vision-Language-Action (VLA) policies. Starting from a pretrained VLA policy, LWD closes the loop between deployment, shared physical experience, policy improvement, and redeployment by using autonomous rollouts and human interventions collected across a robot fleet. To stabilize learning from heterogeneous, sparse-reward fleet data, LWD combines *Distributional Implicit Value Learning* (DIVL) for robust value estimation with Q-learning via *Adjoint Matching* (QAM) for policy extraction in flow-based VLA action generators. We validate LWD on a fleet of 16 dual-arm robots across eight real-world manipulation tasks, including semantic grocery restocking and 3–5 minute long-horizon tasks. A single generalist policy improves as fleet experience accumulates, reaching an average success rate of 95%, with the largest gains on long-horizon tasks.

I. INTRODUCTION

Deploying general-purpose robots in the real world requires *high-performance generalist* policies: policies that can reliably complete a broad range of tasks across diverse objects, environments, user instructions, and operating conditions. Recent Vision-Language-Action (VLA) policies [1, 2, 3, 4, 5, 6] provide a strong foundation by acquiring broad competence from large offline robot datasets. However, offline pretraining alone does not make a policy deployment-ready. Real-world deployment is not a fixed test distribution: as robots are used across more homes, stores, workspaces, and users, they encounter new tasks, object instances, configurations, preferences, and rare failure modes beyond the coverage of pretraining data. Obtaining high performance therefore requires policies that continue to improve from deployment experience, so that adaptation scales with the data generated by use.

This perspective recasts deployment from an endpoint of training into a source of continual policy improvement. Real-

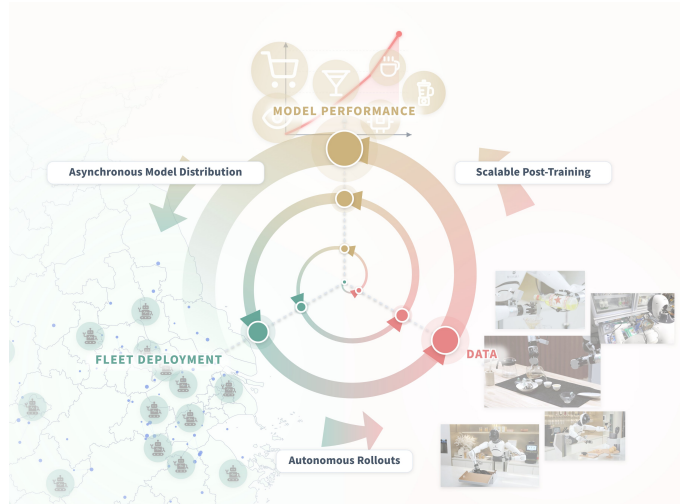


Fig. 1: **Learning While Deploying (LWD): Fleet-scale Reinforcement Learning for Generalist Robot Policies.** A pretrained Vision-Language-Action (VLA) model is first initialized with *human-collected* offline data. The data flywheel then spins up. The model is deployed across diverse real-world robot tasks and *autonomously* collects online interaction data. This online data is mixed with the offline replay buffer to update the model, which is then re-deployed for further data collection.

izing this form of continual improvement requires deployment experience that is both broad and continuously updated. For a generalist robot policy, the most valuable deployment experience is naturally collected at fleet scale. Any individual robot samples only a small portion of the deployed distribution, whereas a fleet spans diverse tasks, environments, objects, and user instructions, producing heterogeneous experience that includes successes, failures, recoveries, partial progress, rare edge cases, and occasional human interventions. Aggregating this physical experience through a shared policy creates a closed-loop data flywheel: deployed robots generate experience on the target deployment distribution, the shared

† Corresponding author.

policy improves from the aggregated data, and the improved policy is redeployed to collect broader and more informative experience.

We refer to this setting as *Learning While Deploying* (LWD): continual policy improvement driven by the accumulated real-world autonomous experience of a deployed robot fleet. Turning this data flywheel into a learning algorithm, however, requires a training objective that can improve from the outcomes of autonomous interaction, rather than treating deployment data for pure imitation signal. Interactive imitation-learning methods [7] can incorporate expert demonstrations, corrections, and interventions during deployment, but they treat deployment primarily as a source of action labels for supervised learning. As a result, they use only part of the available experience and lack a principled mechanism for leveraging autonomous trials that contain successes, failures, recoveries, partial progress, and task rewards. Reinforcement Learning (RL) in principle provides such a mechanism by optimizing policy behavior from task outcomes and policy experience [8, 9, 10, 11]. Yet existing RL approaches for robotics are often limited to small-scale, short-horizon, or task-specific settings, and frequently specialize a pretrained generalist policy to a narrow task [12, 13, 14]. A scalable method for post-training end-to-end VLA policies from fleet deployment experience while preserving their generality remains an open problem.

Addressing this gap requires an RL algorithm for LWD that is compatible with pretrained VLA policies, can learn from large offline and off-policy datasets, and can adapt rapidly as new deployment data streams in. These requirements stress both components of an RL method. Value learning must produce reliable estimates from heterogeneous off-policy data with sparse rewards and rare high-return trajectories. Policy extraction must turn the learned values into better actions from a large generative VLA policy without destabilizing the model.

Prior work addresses these requirements only in part. Amin et al. [15] combines offline value learning with iterative offline RL, but the procedure is slow and does not directly use action gradients from the learned value function. Luo et al. [16, 17] show that online RL can learn challenging robotic manipulation tasks within a short period through real-world interaction, but train task-specific policies from scratch rather than improve a pretrained generalist policy. On-policy VLA finetuning methods [18, 19, 20, 21, 22] update pretrained policies directly from online rollouts, but are not designed to efficiently reuse large offline or off-policy deployment buffers. They also do not learn an explicit action-value critic, and therefore cannot use action-space critic gradients to guide policy improvement. Together, these limitations motivate an offline-to-online RL approach that can reuse heterogeneous deployment data while stably improving a pretrained generative VLA policy.

We present Fleet-Scale Offline-to-Online RL, an offline-to-online framework for post-training end-to-end VLA policies in a large-scale real-world deployment system. The framework couples two pieces: distributional value learning from offline

and autonomous deployment experience, and stable policy extraction that transfers value improvement into a flow-based VLA policy.

For value learning, we introduce Distributional Implicit Value Learning (DIVL). DIVL builds on the value-learning component of Implicit Q-Learning [23], but replaces scalar expected value regression with a distributional value model. This choice is important in the setting of fleet deployment: robots collect data asynchronously under different policy versions, across heterogeneous scenes, with sparse rewards, failures, partial recoveries, and occasional human interventions. As a result, the return associated with the same state-action pair can be multi-modal and heavy-tailed. A scalar critic may collapse these outcomes into an average value and obscure rare but reproducible successes, whereas a distributional critic can preserve these high-return modes. DIVL therefore learns multi-step return distributions while retaining the in-support policy improvement property of implicit value learning. This yields a stable learning signal from large off-policy deployment buffers without requiring the policy to query out-of-distribution actions.

For policy extraction, we adopt Q-learning with Adjoint Matching (QAM) [24, 25]. The critic provides useful action gradients, but backpropagating them through the full multi-step denoising process of a flow policy is unstable and expensive. QAM converts the critic gradient at the denoised action into step-wise supervision for the flow model. This gives a stable way to update the VLA policy from the learned value function while preserving the expressivity of generative action modeling.

The full system has two stages: offline pretraining on a mixture of data from diverse sources, followed by rapid online finetuning with deployment data. Both stages optimize the same RL objective, which mitigates a common offline-to-online mismatch: offline critics can become overly conservative and poorly calibrated for subsequent online finetuning, while online improvement depends on extrapolating values to newly visited actions [26]. We instantiate it on a fleet of 16 dual-arm robots across eight manipulation tasks. These include long-horizon precision tasks, such as brewing Gongfu tea, making cocktails, and making fruit juice, which typically require 3–5 minute executions, as well as shorter-horizon tasks that require semantic generalization, such as restocking diverse items in grocery stores. A single generalist policy trained with LWD improves as online fleet experience accumulates. It substantially improves over the pretrained model, reaches an average success rate of 0.95 across all tasks, and outperforms relevant baselines by large margins. The performance gap is especially pronounced on long-horizon tasks, where RL can propagate rewards through multi-step dynamic programming and stitch together value estimates across partial progress, while imitation-learning methods suffer more severely from compounding errors. This LWD procedure typically requires only a few hours of real-world interaction.

Our main contribution is a fleet-scale offline-to-online RL system for post-training generalist robot policies in real-world

deployment. Algorithmically, LWD combines distributional implicit value learning with QAM-based policy extraction and uses the same RL objective across offline pretraining and online finetuning. Systemically, it enables a distributed robot fleet to aggregate physical interaction experience and autonomously improve a shared VLA policy. To the best of our knowledge, LWD is among the first real-world RL systems to close this offline-to-online improvement loop for generalist robot policies. More broadly, it provides a concrete step toward deploying general-purpose robots at scale: fleet-scale deployment can itself become a source of training data, creating a data flywheel in which deploying more robots improves the shared policy and, in turn, future deployment.

II. RELATED WORK

LWD is a post-training framework for generalist robot policies, instantiated as a distributed large-scale RL system deployed in real-world settings. Accordingly, we survey prior work in the following areas.

A. Post-Training of Robot Generalist Policies

Robot generalist policies, including VLA models, acquire broad capabilities through large-scale pre-training on diverse multi-modal data [2, 3, 4, 6]. To adapt these policies to downstream deployments, recent work has explored several post-training strategies [27, 20, 12, 15, 28, 29]. One direction studies offline post-training, where policies are improved using previously collected rollouts [27, 15, 30]. $\pi_{0.6}^*$ combines offline value learning with iterative offline RL, achieving substantial gains on individual real-world tasks [15]. RLDG uses specialist RL to generate data for policy distillation, providing another way to incorporate RL supervision [30]. However, offline-only post-training follows a collect-train-deploy cycle and cannot immediately incorporate experience gathered during deployment, making adaptation to distribution shifts slow [15, 30]. LWD instead updates the policy during deployment, allowing newly collected experience to correct such shifts quickly.

Another line of work performs post-training with online RL, including VLA-RL [18] and RIPT [19], achieving strong improvements for specialist policies in simulated tasks [31, 32, 33, 34, 20, 35, 36]. However, these methods typically rely on on-policy data collection, which can be sample-inefficient and costly for real-world robots [20, 37]. In contrast, LWD learns from large offline datasets together with off-policy online replay, improving the practicality of real-world post-training.

Recent methods also combine offline and online phases: offline pretraining on rollout datasets followed by online refinement through real-time interactions [13, 14, 12]. However, prior methods typically learn specialist policies tailored to individual tasks, limiting generalization across diverse deployments [13, 12].

LWD is fundamentally different from these works: it performs offline-to-online post-training for a generalist robot policy rather than learning task-specific specialists. This enables

scalable post-training of a single policy across multiple real-world tasks, including long-horizon tasks with sparse rewards.

B. Offline-to-Online Reinforcement Learning

Offline-to-online RL pretrains on diverse offline data and refines continuously through online interactions [38, 25, 39, 40, 12, 14, 30, 41, 42]. Luo et al. [16, 17] utilize a small number of human demonstrations to seed policy learning and then specialized a single robotic skill through real-world interaction. However, LWD differs from this method in that it post-trains a shared generalist VLA policy across multiple tasks, combines offline and online replay in one learning loop, and operates under distributed fleet-scale deployment. Recent studies use different policy-extraction mechanisms to reuse offline data during online improvement [25, 43, 44, 45]. Wagenmaker et al. [45] present DSRL, which adapts pretrained diffusion policies via RL on latent-noise space for sample-efficient online-to-offline improvement. Li and Levine [25] introduce QAM, using critic gradients to improve flow-based policies through adjoint matching, achieving stable training from scratch in simulation. However, prior approaches have not been validated for stable, fleet-scale post-training of generalist VLA policies. LWD addresses this and adopts QAM to enable offline-to-online RL through large-scale real-world deployments.

Recent robotic post-training methods incorporate offline-to-online RL to improve policies [13, 12, 14, 30]. However, they typically focus on task-specific policies with inconsistent training objectives across offline-to-online stages, and operate at limited deployment scale. In contrast, LWD trains a generalist policy across diverse tasks through fleet-scale offline-to-online RL. It adopts a unified training method in offline and online stages, enhancing training stability and scalability.

C. Large-Scale Robotic RL Systems

Large-scale robotic RL systems improve robot policies by aggregating experience from distributed actors and training centralized learners, enabling policy improvement beyond what can be achieved with isolated task-level data collection [46, 47, 48, 49, 50, 51, 52]. Kalashnikov et al. [46, 47] demonstrate that off-policy RL can be scaled from vision-based grasping to multi-task manipulation through asynchronous robot data collection and centralized Q-function optimization. While these systems focus primarily on short-horizon manipulation and learn policies largely from scratch, LWD post-trains a pretrained generalist VLA policy across diverse real-world tasks, including long-horizon manipulation. Bousmalis et al. [50] and Herzog et al. [52] further study learning from large-scale robot experience, but the former relies on behavior cloning while the latter targets task-specific RL for waste sorting. Most recently, SOP Pan et al. [49] formalizes the system substrate for scalable online post-training of VLA policies, coupling a distributed robot fleet with a centralized cloud learner and asynchronous policy synchronization. Building on this deployment substrate, LWD instantiates the learning algorithm with offline-to-online RL:

it jointly leverages prior offline data and newly collected fleet experience to improve a single generalist policy across long-horizon real-world tasks.

This distinction shifts the contribution from distributed execution alone to an RL-driven data flywheel, where large-scale deployment continually supplies experience for policy improvement.

III. PRELIMINARIES

A. Problem Setting and Notation

We formulate robot control as a Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, r, \gamma)$, where $\gamma \in (0, 1]$ is the discount factor. We consider a set of tasks indexed by $k \in \mathcal{K}$. Each state $s = (o, \ell_k) \in \mathcal{S}$ consists of a robot observation o and a language instruction ℓ_k specifying task k . For long-horizon tasks, ℓ_k is a high-level command, such as ‘Make Tea’ rather than a sequence of low-level subtask instructions. In our setting, we use sparse binary rewards, with $r = 1$ only when an episode terminates successfully and $r = 0$ otherwise.

LWD trains one shared generalist VLA policy across all tasks. At time t , the policy takes the state s_t as input and outputs an action chunk

$$\mathbf{a}_t \equiv \mathbf{a}_{t:t+H} = [a_t, a_{t+1}, \dots, a_{t+H-1}] \sim \pi_\theta(\cdot | s_t), \quad (1)$$

which is executed before replanning. The corresponding chunk reward is

$$\mathbf{r}_t \equiv \mathbf{r}_{t:t+H} = \sum_{i=0}^{H-1} \gamma^i r_{t+i}. \quad (2)$$

Thus, mixed-task replay samples are written abstractly as $(s_t, \mathbf{a}_t, \mathbf{r}_t, s_{t+H}) \sim \mathcal{D}$, where $\mathcal{D}$ denotes the replay distribution. In the offline stage, $\mathcal{D}$ is induced by samples from $\mathcal{B}_{\text{off}}$; in the online stage, it is induced by mixed replays from $\mathcal{B}_{\text{off}} \cup \mathcal{B}_{\text{on}}$. Throughout the method, the generalist policy and critic operate on action chunks.

B. Implicit Q-Learning

Implicit Q-Learning (IQL) [23] avoids explicit action maximization by fitting a *scalar* state-value function to a high *expectile* of dataset action-values. Using the chunk notation from above, with $\mathbf{a}_t$ and $\mathbf{r}_t$, IQL fits

$$\mathcal{L}_V^{\text{IQL}}(\psi) = \mathbb{E}_{\mathcal{D}} \left[\rho_{\tau, 2}(Q_{\bar{\phi}}(s_t, \mathbf{a}_t) - V_\psi^{\text{IQL}}(s_t)) \right], \quad (3)$$

where

$$\rho_{\tau, 2}(u) = |\tau - \mathbb{I}(u < 0)| u^2, \quad (4)$$

and $Q_{\bar{\phi}}$ denotes the target network, whose parameters are updated by exponential moving average. The critic Q_ϕ is trained with the value-based TD target

$$y_t^{\text{IQL}} = \mathbf{r}_t + \gamma^H V_\psi^{\text{IQL}}(s_{t+H}), \quad (5)$$

using

$$\mathcal{L}_Q^{\text{IQL}}(\phi) = \mathbb{E}_{\mathcal{D}} \left[\left(Q_\phi(s_t, \mathbf{a}_t) - y_t^{\text{IQL}} \right)^2 \right]. \quad (6)$$

For $\tau > 1/2$, this value estimate is biased toward higher-valued dataset actions, giving an implicit improvement target

without a $\max_{\mathbf{a}} Q(s, \mathbf{a})$ backup. In LWD, we retain this *asymmetric bootstrap* principle, but replace scalar expectile value regression with a *distributional* value model and *quantile*-based value extraction.

C. Flow Matching and Q-learning with Adjoint Matching

Flow Matching (FM) [53] represents a generative policy as a time-dependent vector field. Given a data action chunk $\mathbf{a}^1 = \mathbf{a}$ and Gaussian noise $\mathbf{a}^0 \sim \mathcal{N}(0, I)$, FM defines the interpolation

$$\mathbf{a}^w = (1 - w)\mathbf{a}^0 + w\mathbf{a}^1, \quad w \in [0, 1], \quad (7)$$

and trains a conditional vector field $f_\theta(s, \mathbf{a}^w, w)$ to match the velocity $\mathbf{a}^1 - \mathbf{a}^0$. Flow-based VLA policies use this construction as an action-generation head [5, 6].

For policy extraction, a flow policy must be optimized through a multi-step generation process, making direct critic backpropagation costly and potentially unstable. Q-learning with Adjoint Matching (QAM) [25] addresses this problem by combining TD critic learning with an adjoint-matching policy update. Given a pretrained reference flow f_β and a critic Q_ϕ , QAM defines the KL-regularized improvement target

$$\pi^*(\mathbf{a} | s) \propto \pi_\beta(\mathbf{a} | s) \exp(Q_\phi(s, \mathbf{a})/\lambda), \quad (8)$$

where λ is the temperature. The resulting policy update can be written as a local regression objective along trajectories of the reference flow:

$$\begin{aligned} f_\delta(s, \mathbf{a}^w, w) &= f_\theta(s, \mathbf{a}^w, w) - f_\beta(s, \mathbf{a}^w, w) \\ \mathcal{L}_{\text{QAM}}(\theta) &= \mathbb{E} \left[\int_0^1 \left\| \frac{2f_\delta(s, \mathbf{a}^w, w)}{\sigma_w} + \sigma_w \tilde{g}_w \right\|_2^2 dw \right], \end{aligned} \quad (9)$$

where $\sigma_w = \sqrt{2(1-w)w}$ and $\tilde{g}_w$ is the adjoint state with terminal condition

$$\tilde{g}_1 = -\nabla_{\mathbf{a}} [Q_\phi(s, \mathbf{a}^1)/\lambda]. \quad (10)$$

LWD uses QAM [25] as its policy-extraction mechanism, using the critic learned from DIVL to form local regression targets for the flow policy.

IV. LEARNING WHILE DEPLOYING

LWD follows the offline-to-online procedure shown in Fig. 2(a). The offline stage trains the policy, critic, and distributional value model on a static replay buffer $\mathcal{B}_{\text{off}}$, providing the initialization for deployment. The online stage deploys the current policy to a fleet of robot actors for autonomous rollouts, which populate $\mathcal{B}_{\text{on}}$ with policy transitions and optional human interventions. The learner updates V_ψ , Q_ϕ , and f_θ on mixed replay from $\mathcal{B}_{\text{off}} \cup \mathcal{B}_{\text{on}}$, and periodically redeploys the updated policy. This forms a data flywheel: robot rollouts expand replay, mixed replay updates the policy, and refreshed checkpoints are redeployed to the fleet.

This procedure contains two key algorithmic components. First, *Distributional Implicit Value Learning (DIVL)* trains the critic Q_ϕ and the distributional value model V_ψ for value learning. Second, QAM-based policy extraction updates the flow policy f_θ using the action gradient of Q_ϕ learned from DIVL.

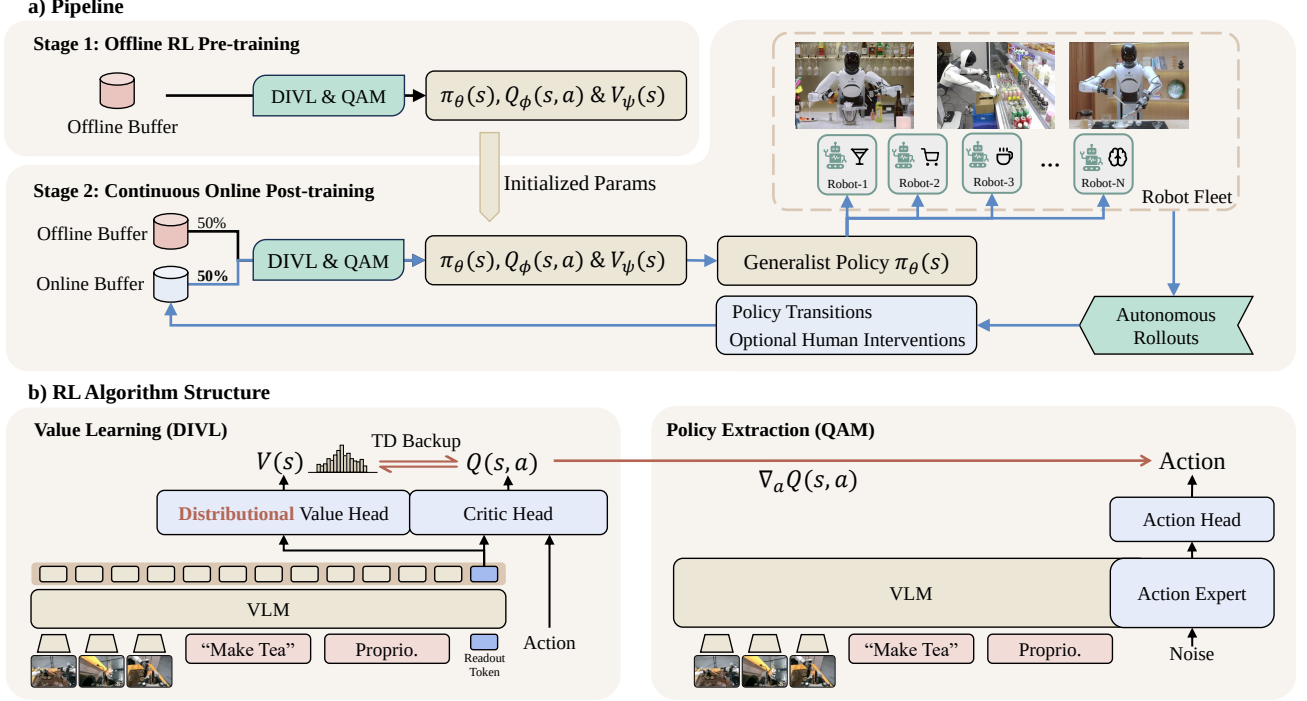


Fig. 2: **LWD overview.** (a) **Pipeline.** Training is organized into two stages. Stage 1 performs offline RL pre-training on an offline buffer. Stage 2 conducts continuous online post-training with mixed replay from both the static offline buffer and a continuously updated online buffer. A fleet of actors is autonomously deployed on diverse real-world robot tasks to collect online data and appends it to a continually updated online buffer. (b) **Algorithm structure.** A VLM-based model $\pi_\theta(s)$ maps states to action chunks through a policy head, which is optimized with QAM loss for policy extraction. In parallel, a critic $Q_\phi(s, a)$ and a distributional value $V_\psi(s)$ are trained with TD losses for value learning.

A. Distributional Implicit Value Learning

Distributional Implicit Value Learning (DIVL) is the value-learning component of LWD. It learns a distribution over replay action-values and uses a quantile of this distribution as the bootstrap target for the chunk-level critic $Q_\phi(s_t, \mathbf{a}_t)$. This design keeps the asymmetric bootstrap principle of IQL [23] while avoiding a single scalar expectile target.

Concretely, the distributional value model $V_\psi(s_t)$ represents the state-conditioned distribution of dataset action-values [54]:

$$p_\psi(v | s_t) = P(v = Q_\phi(s_t, \mathbf{a}_t) | \mathbf{a}_t \sim \mathcal{D}(\cdot | s_t)). \quad (11)$$

where $\mathcal{D}(\cdot | s_t)$ denotes the empirical replay action distribution conditioned on s_t . Thus, $V_\psi(s_t)$ is not a scalar value estimate. Instead, it represents the distribution of scalar critic values assigned to replay actions at state s_t .

We fit this distribution by minimizing the negative log-likelihood of scalar critic targets from the exponential-moving-average (EMA) critic Q_ϕ :

$$\mathcal{L}_V(\psi) = \mathbb{E}_{(s_t, \mathbf{a}_t) \sim \mathcal{D}} \left[-\log p_\psi(Q_\phi(s_t, \mathbf{a}_t) | s_t) \right]. \quad (12)$$

In our implementation, p_ψ is represented as categorical discretization; Appendix A1 gives the details.

Compared with scalar regression used in IQL, such distributional parameterization is better matched to LWD. Prior

work [55] finds that a categorical distributional representation of return values is helpful in diverse multi-task offline RL settings. Moreover, it supports two designs used below: the bootstrap statistic is selected as a quantile of $p_\psi(v | s_t)$ without refitting a scalar value function, and the entropy of $p_\psi(v | s_t)$ provides the uncertainty signal for adapting τ .

We use the τ -quantile of $V_\psi(s_t)$ as the bootstrap statistic:

$$\text{Quant}_\tau(V_\psi(s_t)) \triangleq \inf \{v : F_\psi(v | s_t) \geq \tau\}. \quad (13)$$

where $F_\psi(v | s_t)$ be the cumulative distribution function induced by $p_\psi(v | s_t)$. This yields the TD target

$$y_Q = \mathbf{r}_t + \gamma^H \text{Quant}_\tau(V_\psi(s_{t+H})), \quad (14)$$

and the critic loss

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s_t, \mathbf{a}_t, \mathbf{r}_t, s_{t+H}) \sim \mathcal{D}} \left[(Q_\phi(s_t, \mathbf{a}_t) - y_Q)^2 \right]. \quad (15)$$

The τ -quantile is an in-distribution optimistic bootstrap statistic over replay actions, rather than an explicit max backup over the full action space. This fits the offline RL setting, where the target should favor high-value replay actions without extrapolating aggressively beyond the data. IQL addresses the same issue with scalar expectile value regression. DIVL keeps this asymmetric value-learning principle, but realizes it through a distributional model and a quantile statistic.

To make this connection explicit, we write the value target under a generalized asymmetric loss family:

$$\rho_{\tau,p}(u) = |\tau - \mathbb{I}(u < 0)| \cdot |u|^p, \quad (16)$$

where $p = 2$ gives the expectile form used by IQL and $p = 1$ gives the quantile form used by DIVL.

Proposition 1 (Distributional view of asymmetric value learning): For any fixed asymmetric loss in this family, direct scalar regression and our two-step procedure of fitting the value distribution and extracting the corresponding asymmetric statistic yield the same optimal scalar value.

See Appendix A2 for proof. The proposition shows that DIVL’s two-step procedure of distributional value estimation and τ -quantile extraction has the same optimum as the corresponding direct asymmetric value regression objective.

This result supports using a quantile of the learned value distribution as the bootstrap target for a fixed τ . The value of τ controls the optimism of this target: larger values select higher quantiles and give more optimistic targets, while smaller values give more conservative ones. In mixed-task replay, the same level of optimism is not appropriate for every state, so we adapt τ using uncertainty in the learned value distribution.

Specifically, we use the normalized entropy of the categorical distribution $p_\psi(\cdot | s_{t+H})$ as the uncertainty signal:

$$\mathcal{H}(s_{t+H}) = -\frac{1}{\log C} \sum_{c=1}^C p_{\psi,c}(s_{t+H}) \log p_{\psi,c}(s_{t+H}), \quad (17)$$

where C is the number of categories and $p_{\psi,c}(s_{t+H})$ is the probability assigned to category c . Then, the adaptive schedule is

$$\tau(s_{t+H}) = \text{clip}(\tau_{\text{base}} - \alpha \mathcal{H}(s_{t+H}), \tau_{\min}, \tau_{\max}), \quad (18)$$

where τ_{base} is the target for confident states, $\alpha \geq 0$ controls uncertainty sensitivity, and the hyperparameter values are reported in Appendix B2. Diffuse distributions receive lower τ values to reduce overestimation, while concentrated distributions retain more optimistic targets. We treat $\tau(s_{t+H})$ as stop-gradient when computing the TD target.

B. Policy Extraction via QAM

Policy extraction in LWD starts from a pretrained flow-matching VLA and aims to improve its action distribution using the DIVL critic. Existing offline RL methods often extract a policy without differentiating through Q_ϕ , for example by advantage-weighted regression on replay actions [56, 43, 23, 57]. This update is poorly matched to flow-based VLA policies, since it requires evaluating the log likelihood of action chunks under the multi-step denoising process of the flow policy. More generally, the KL-regularized policy improvement target has a Boltzmann form, whose normalizer requires integrating over high-dimensional action chunks.

An alternative is to use the first-order action gradient $\nabla_{\mathbf{a}} Q_\phi(s, \mathbf{a})$ to improve sampled action chunks. For flow policies, however, applying this update via direct backpropagation

through the full multi-step generation process is computationally expensive and numerically unstable (see Appendix A3 for analysis). This makes direct critic backpropagation difficult to use as the optimization method for large VLA policies.

We therefore use QAM for policy extraction [25] as shown in the right of Fig. 2(b). As outlined in Section III-C, QAM reformulates trajectory-level policy optimization into a local regression objective along the reference flow. Specifically, the DIVL critic Q_ϕ supplies the reward-informed gradient to initialize the terminal adjoint state $\tilde{g}_1$, which in turn guides the refinement of the policy vector field (Eq. (10)).

In particular, we keep f_β fixed as the behavior-cloned flow initialized before offline RL, and optimize f_θ throughout both offline and online training. For each replay minibatch, as shown in lines 5–7 of Algorithm 2, we sample states and Gaussian noise, roll out f_β to generate reference flow trajectories, evaluate $\nabla_{\mathbf{a}} Q_\phi(s, \mathbf{a})$ at the generated endpoint, solve the adjoint dynamics, and regress f_θ toward the resulting local targets.

C. Offline to Online RL Training Pipeline

Following the LWD loop in Fig. 2(a) as introduced in the opening of Section IV, post-training proceeds in two stages that share the same value-learning and policy-extraction objectives but differ in data source.

The offline stage trains on an offline buffer $\mathcal{B}_{\text{off}}$ as shown in the Stage 1 of Fig. 2(a) and lines 4–7 of Algorithm 1. This offline buffer contains three sources: *demonstrations*, expert-collected successful trajectories; *rollouts*, generated by historical policies, including both successes and failures; and *play data*, consisting of human-guided exploration of failure modes. All three sources are converted into the same chunked transition format as online replay, with terminal success or failure labels used to assign sparse binary rewards. The details of data structure are shown in Table IV. LWD applies the offline buffer to pre-train the policy π_θ , critic Q_ϕ and distributional value V_ψ , providing a strong initialization for deployment and training in the online stage.

Moreover, since long-horizon tasks last thousands of steps and have extremely sparse rewards, the one-step target in Eq. (14) can propagate success signals slowly. We therefore use an n -step chunk-level TD target in the offline stage to cold-start the critic and distributional value model:

$$y_Q = \sum_{i=0}^{n-1} \gamma^{iH} \mathbf{r}_{t+iH} + \gamma^{nH} \text{Quant}_{\tau(s_{t+nH})}(V_\psi(s_{t+nH})), \quad (19)$$

where $n = 1$ for short tasks such as grocery restocking tasks and $n = 10$ for long-horizon tasks. If an episode terminates within the n -step window, we truncate the return at the terminal chunk and remove the bootstrap term. This target accelerates sparse reward propagation through the fixed offline replay buffer. During online training, we found long multi-step targets less effective. One possible reason is that online trajectories mix policy transitions with human interventions. Longer backups are more likely to cross these sources, so the TD path

Algorithm 1 LWD: Offline-to-Online Training Pipeline

Require: offline buffer $\mathcal{B}_{\text{off}}$; demonstration dataset $\mathcal{D}_{\text{demo}} \subset \mathcal{B}_{\text{off}}$; online buffer $\mathcal{B}_{\text{on}}$; robot actor fleet $\mathcal{F}$; offline budget N_{off} ; online budget N_{on} ; actor-sync period N_{sync} .

- 1: Pretrain policy $\pi_\theta \leftarrow \mathcal{D}_{\text{demo}}$
- 2: Set fixed reference policy $\pi_\beta \leftarrow \pi_\theta$
- 3: Initialize Q_ϕ, V_ψ ; set target $Q_{\bar{\phi}} \leftarrow Q_\phi$
 // Stage 1: Offline Pretraining
- 4: **for** $i \leftarrow 1 : N_{\text{off}}$ **do**
- 5: Sample mini-batch $\mathcal{B}^{\text{mini}} \sim \mathcal{B}_{\text{off}}$
- 6: $(Q_\phi, V_\psi, \pi_\theta, Q_{\bar{\phi}}) \leftarrow \text{LEARNER}(\mathcal{B}^{\text{mini}}; Q_\phi, V_\psi, \pi_\theta, \pi_\beta, Q_{\bar{\phi}})$
- 7: **end for**
 // Stage 2: Continuous Online Training
- Robot actor process (Asynchronously):**
- 8: Deploy π_θ to each robot from $\mathcal{F}$
- 9: **while** online training is active **do**
- 10: done $\leftarrow \text{False}$; $T \leftarrow 0$
- 11: **while** not done **do**
- 12: Execute $\mathbf{a} \leftarrow \pi_\theta(s)$ until done
- 13: **if** intervention is required **then**
- 14: Human intervenes $\mathbf{a} \leftarrow \mathbf{a}_H$
- 15: **end if**
- 16: $s' \leftarrow \text{UpdateObs}(s, \mathbf{a})$
- 17: done $\leftarrow \mathbb{I}[\text{TimeLimit} \vee \text{Failure} \vee \text{Success}]$
- 18: $r \leftarrow \mathbb{I}[\text{done} \wedge \text{Success}]$
- 19: $T \leftarrow T + 1$
- 20: **end while**
- 21: $\mathbf{r} \leftarrow \text{UpdateChunkedReward}(r)$
- 22: $\mathcal{B}_{\text{on}} \leftarrow \mathcal{B}_{\text{on}} \cup \{(s_t, \mathbf{a}_t, \mathbf{r}_t, s'_{t+H})\}$
- 23: $\pi_\theta \leftarrow \text{FetchNewPolicy}(\pi_\theta^{\text{new}})$
- 24: **end while**
- Central learner process (Asynchronously):**
- 25: **for** $j \leftarrow 1 : N_{\text{on}}$ **do**
- 26: Sample mini-batch $\mathcal{B}^{\text{mini}} \sim \{\mathcal{B}_{\text{off}} \cup \mathcal{B}_{\text{on}}\}$
- 27: $(Q_\phi, V_\psi, \pi_\theta, Q_{\bar{\phi}}) \leftarrow \text{LEARNER}(\mathcal{B}^{\text{mini}}; Q_\phi, V_\psi, \pi_\theta, \pi_\beta, Q_{\bar{\phi}})$
- 28: **if** $j \bmod N_{\text{sync}} = 0$ **then**
- 29: Deploy latest policy π_θ to each robot from $\mathcal{F}$
- 30: **end if**
- 31: **end for**
- 32: **return** $Q_\phi, V_\psi, \pi_\theta$

may not correspond to a single policy execution. Since the critic and value model already have offline initialization, we therefore use 1-step chunk-level TD targets for online updates.

The online stage deploys the offline-initialized policy to the robot fleet, as shown in Stage 2 of Fig. 2(a) and lines 8–31 of Algorithm 1. Robots execute the current policy checkpoint and asynchronously stream *policy transitions* into an online buffer $\mathcal{B}_{\text{on}}$. When a rollout requires correction, as judged by human, the operator may intervene. Intervention segments are stored in $\mathcal{B}_{\text{on}}$ as regular online replay transitions with the executed corrective actions, and rewards are assigned using the same terminal success or failure labels as autonomous

Algorithm 2 LEARNER: Single Update of DIVL and QAM

Require: mini-batch $\mathcal{B}^{\text{mini}} = \{(s_t, \mathbf{a}_t, \mathbf{r}_t, s_{t+H})\}$; critic Q_ϕ with target $Q_{\bar{\phi}}$; distributional value V_ψ ; policy π_θ with reference policy π_β ; EMA rate ρ .
 // Distributional Implicit Value Learning

- 1: Update ψ by minimizing Eq. (12)
- 2: Compute TD target y_Q via Eq. (19)
- 3: Update ϕ by minimizing Eq. (15)
- 4: $\bar{\phi} \leftarrow \rho \bar{\phi} + (1 - \rho) \phi$
 // Policy Extraction via QAM
- 5: Sample Gaussian noise $\mathbf{a}_t^0 \sim \mathcal{N}(0, I)$
- 6: Roll out the reference trajectory $\{\mathbf{a}_t^w\}_{w \in [0,1]}$ via π_β
- 7: Set the endpoint $\mathbf{a}_t^1 = \mathbf{a}_t$
- 8: Update θ by minimizing Eq. (9) with $\tilde{g}_1$ set from action gradient $\nabla_{\mathbf{a}} Q_\phi(s, \mathbf{a}_t^1)$ via Eq. (10)
- 9: **return** $(Q_\phi, V_\psi, \pi_\theta, Q_{\bar{\phi}})$

rollouts. Thus, online replay contains both autonomous policy transitions and human-intervention transitions [17]. Online Training continues with the same value-learning and policy-extraction objectives on mixed replay from $\mathcal{B}_{\text{off}} \cup \mathcal{B}_{\text{on}}$, while updated policy checkpoints are periodically published back to the robots.

D. Architectures

Fig. 2(b) shows the concrete neural network architecture used by LWD. The policy and value/critic networks are separate modules, isolating action generation from value and critic optimization. Only the policy checkpoint is asynchronously distributed to the robot fleet for inference, while the value and critic networks remain on the centralized learner.

We implement V_ψ and Q_ϕ with a shared Gemma3–SigLIP VLM backbone and separate prediction heads. The Gemma 3 language module and SigLIP vision encoder are initialized from publicly released Gemma 3-270M-IT [58] and SigLIP-So400M checkpoints [59], while the visual projection layer and value/critic heads are initialized from scratch.

Following the use of readout tokens as compact transformer representations [60, 61, 62], we apply the shared backbone to the multimodal sequence for state s_t and denote the final hidden state of the readout token by z_t , which serves as the state representation for both value and critic prediction. The value head predicts logits over a fixed categorical support. Following the C51 projection [54], the scalar supervision target $Q_{\bar{\phi}}(s_t, \mathbf{a}_t)$ is clipped to the value support and linearly projected onto its two neighboring atoms, yielding a target distribution m_t .

The critic conditions on both the state representation z_t and the action chunk $\mathbf{a}_t$. The action chunk is encoded with a learned temporal attention pooling layer and concatenated with z_t . The resulting representation is fed into two scalar critic heads in a clipped double-Q design, where the minimum critic estimate is used for DIVL target construction and TD

backups to mitigate overestimation.

The actor follows the $\pi_{0.5}$ flow-based VLA architecture [6]. It consists of a PaliGemma vision-language backbone, instantiated with a Gemma-2B language model and a SigLIP vision encoder, together with a Gemma-300M action expert for flow-based action generation.

In the offline RL stage, both the actor and the value/critic networks are fully fine-tuned; the resulting weights initialize online training. During online QAM updates, the policy VLM backbone is frozen and only the action expert is updated, while the value and critic networks continue to be fully fine-tuned on mixed replay. This design keeps online policy updates efficient and preserves the pretrained vision-language representations, while allowing the value and critic networks to adapt to the evolving replay distribution and provide updated policy-improvement signals.

V. EXPERIMENTAL EVALUATIONS

We evaluate LWD on eight real-world manipulation tasks including grocery stocking and long-horizon manipulation tasks such as tea making, juice making and more. We compare against the reference policy, SFT [53] and two representative post-training baselines, RECAP [15] and HG-Dagger [7]. Our experiments seek answers to whether deployment-time online updates from a shared robot fleet improve over static or offline policies in the same task setting; how LWD compares with the baselines; whether the learned value function provides a useful progress signal under sparse terminal rewards, and which design choice of DIVL contributes to the observed gains. The main results addresses method performance, the value-function visualization diagnoses whether sparse-reward value estimates track task progress, and the ablations isolate the DIVL value-estimation design choices.

A. Experimental Setup

1) Tasks, Evaluation, and Robots:

a) Tasks: We evaluate LWD on eight real-world tasks, as shown in Fig. 3. The grocery restocking tasks consist of four distinct tasks: flat-shelf restocking, misplaced-item correction, freezer restocking with door operation, and open-cooler restocking with carton handling. Together, they test the policy’s ability to follow language instructions and generalize semantically across realistic store scenarios. In each task, the robot must identify the object specified by the instruction among cluttered candidates, handle variations in shelf layout and container geometry, and complete the required placement. The evaluation varies object instances, clutter, shelf and container layouts, language instructions, and store configurations.

We also evaluate our methods on four long-horizon tasks: brewing Gongfu Tea, making Fruit Juice, making Cocktail, and packing shoes into a Shoebox. Each episode typically lasts 3–5 minutes and contains 5–8 annotated subtasks, creating long-range dependencies across planning, manipulation, and recovery. Success requires stable multi-stage executions with precise contact-rich skills, including grasp adjustment, container handling, pouring, tool use, and final placement. Evaluation

episodes include natural reset variability in object poses, tool locations, ingredients, scene initialization, perturbations, and occasional retry or recovery situations.

b) Evaluation metrics: We report task-level scores for all tasks, with different scoring protocols for the two task groups. For the grocery restocking tasks, we follow the protocol of SOP [49]: an episode is successful if the robot follows the right language instruction and task completion within the time limit, yielding a binary success rate. For long-horizon tasks, we report a step-wise success score. Each annotated sub-step is scored as 1 (fully autonomous success), 0.5 (success with minor imperfection, or success with a single retrieval), or 0 (failure after multiple attempts), and the task score is the average across sub-steps. The score is assigned by trained human evaluators according to a predefined rubric that is applied consistently across methods and tasks. We additionally report cycle time on long-horizon tasks to evaluate execution efficiency. Cycle time is computed over both successful and failed attempts, with failed trajectories clipped at predefined task-specific timeout thresholds.

c) Robot fleet setup: All experiments are conducted on the Agibot G1 dual-arm manipulation platform. Each G1 robot has two 7-DoF arms with parallel-jaw grippers and three RGB cameras (one head-view and two wrist-view). The policy runs joint-position control at 30 Hz. As shown in Fig. 4, we deploy a fleet of 16 robots for concurrent rollout collection during online training: 4 robots for the grocery restocking tasks and 3 robots for each long-horizon task. The fleet is connected to a distributed actor-learner system: edge actors upload complete episodes, a centralized learner fetches versioned replay data, and publishes the updated policies to each actor; more details can be seen in Appendix D. For each online experiment, each method is allocated a 4-hour wall-clock budget, corresponding to approximately 60 total hours of online data collected across the robot fleet. Robots collect rollouts asynchronously, and episodes from all tasks are pooled into a single online replay buffer for updating the shared policy. The buffer contains both autonomous rollouts and human intervention segments when intervention is required. The learner broadcasts the updated shared policy to the robot fleet every 50 training steps. Additional training details and hyperparameters are provided in Appendix B2.

2) Baselines and Reference Policies: We compare against two post-training baselines, RECAP and HG-Dagger, and SFT as a reference policy. SFT only utilizes human demonstrations with standard flow-matching loss. RECAP [15] starts from the reference policy and performs iterative post-training on autonomous rollouts. We preserve its advantage-conditioned policy-improvement recipe, but implement it in a multi-task setting. HG-Dagger [7] also starts from the reference policy, then uses online successful rollouts for training. Implementation details and hyperparameters are provided in Appendix C1.

B. Main Results

Table I reports the main quantitative results across all eight real-world tasks. LWD (Online) achieves an average score of

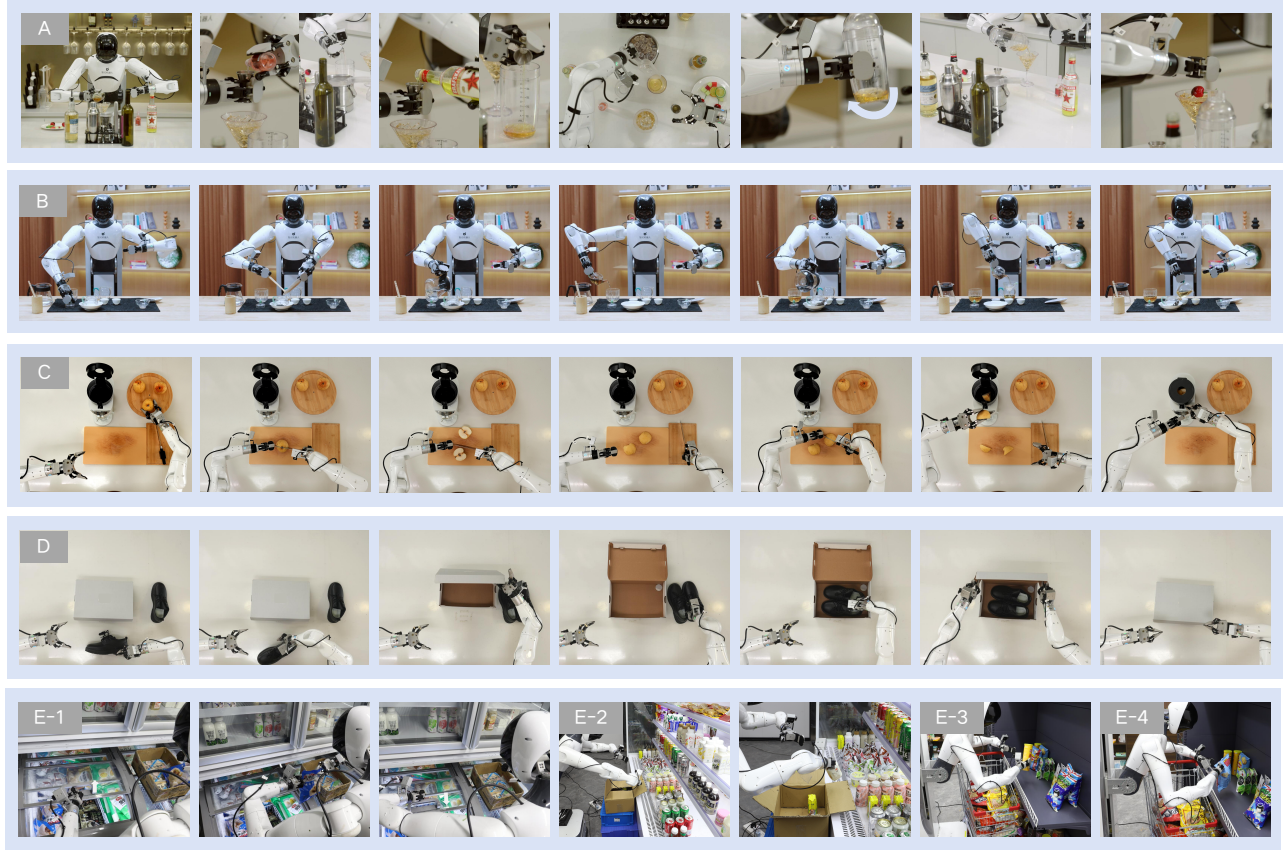


Fig. 3: **Illustrations of our evaluation tasks.** Panels A–D show the four long-horizon tasks, and Panel E summarizes the four grocery restocking tasks. **(A) Make Cocktail:** A sequence of robot manipulation actions for cocktail making: measuring and mixing multiple liquors in a shaker, adding ice, shaking the cocktail, pouring it into a stemmed glass, and garnishing it with a cherry. **(B) Brew Gongfu Tea:** A robot manipulation sequence for Gongfu tea preparation: adding tea leaves, rinsing and draining, brewing with hot water, transferring the tea to a fairness pitcher, distributing it into three teacups, and serving. **(C) Make Fruit Juice:** The sequence for fruit juicing, including cutting and reorienting the fruit, slicing it into pieces, transferring the pieces into a juicer, closing the lid, and rotating the control knob to start juicing. **(D) Pack Shoes:** A manipulation sequence of packing shoes into a shoebox and placing the shoebox neatly. **(E) Grocery Restocking Tasks:** Robot manipulation tasks in various grocery scenarios, including freezer restocking involving door manipulation, open-cooler restocking with carton handling, and flat-shelf restocking with misplacement correction. Together, the suite stresses semantic grounding, contact-rich manipulation, long-horizon execution, and recovery from execution errors.



Fig. 4: **Fleet of robots.** LWD performs online training across a fleet of 16 robots, continually improving a single generalist policy on multiple tasks.

0.95, outperforming all baselines across the evaluated tasks and maintaining strong performance on both short-horizon and long-horizon tasks.

The benefit of LWD is more pronounced on long-horizon tasks. LWD (Online) reaches an average long-horizon step-wise score of 0.91, outperforming SFT (0.68), RECAP (0.77), HG-Dagger (0.73), and LWD (Offline) (0.79). This improvement can be attributed to a consistent offline-to-online RL training pipeline and more complete utilization of available data. LWD (Online) incorporates successful demonstrations, play data, and both successful and failed online trajectories into reward-based policy improvement, enabling the policy to continuously identify and mitigate failure modes encountered during deployment.

LWD (Offline) builds on the reference policy and improves

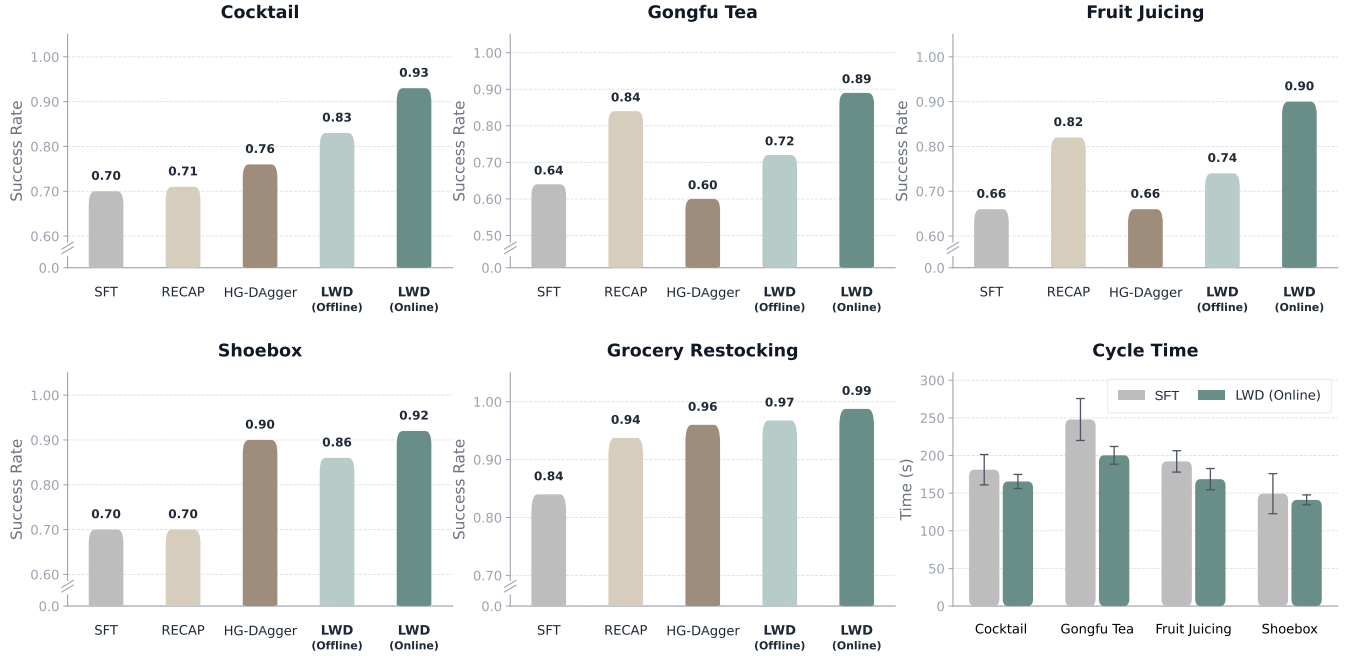


Fig. 5: **Success scores and cycle-time comparison.** LWD achieves higher success scores while reducing mean cycle time relative to the static SFT reference policy. Complete results are shown in Table I.

TABLE I: **Complete results on eight real-world manipulation tasks**, covering four grocery restocking tasks and four long-horizon tasks. We report task success rate for each task (binary success for grocery restocking tasks; average step-wise score across sub-steps for long-horizon Tasks) and the average across all eight tasks in the last column. The best result per column is shown in bold. Our LWD (Online) attains the best overall average (0.95) and achieves the top score on all four long-horizon tasks, while remaining at or near the best on the grocery restocking tasks.

Method	Grocery Restocking Tasks				Long-Horizon Tasks				Average
	Restocking	Correction	Freezer	Open-Cooler	Gongfu Tea	Fruit Juice	Cocktail	Shoebox	
SFT [53]	0.70	0.88	0.83	0.95	0.64	0.66	0.70	0.70	0.76
RECAP [15]	0.95	0.96	0.94	0.95	0.84	0.82	0.71	0.70	0.85
HG-Dagger [7]	1.00	0.92	0.92	1.00	0.60	0.66	0.76	0.90	0.85
LWD (Offline, Ours)	1.00	1.00	0.92	0.95	0.72	0.74	0.83	0.86	0.88
LWD (Online, Ours)	1.00	1.00	0.97	0.98	0.89	0.90	0.93	0.92	0.95

it through offline reinforcement learning. It trains on an offline replay buffer containing successful demonstrations, failed rollouts, and diverse play data, allowing the policy to exploit reward and outcome information beyond imitation-only supervision. We further observe that HG-Dagger yields only limited gains over the reference policy on long-horizon tasks and can even degrade performance on some tasks. A likely reason is that Dagger-style training relies on human correction data, whose variability can introduce inconsistencies and provide limited exploration of the broader state space. In contrast, RL can exploit a wider range of states and directly optimize task-specific rewards. For long-horizon tasks, terminal success signals can be propagated to earlier decision steps through TD backups, improving value estimation across different task stages and providing a stronger learning signal for policy improvement.

On the grocery restocking tasks, all methods except SFT

achieve high scores, leaving limited room for improvement. Even in this saturated regime, LWD (Online) remains at or near the best-performing result on every grocery task. This indicates that LWD provides benefits beyond long-horizon tasks while preserving the generalist behavior of the shared policy during online learning.

Fig. 5 further reports the mean and standard error of cycle time on long-horizon tasks. LWD reduces mean cycle time by 23.75 seconds compared with reference policy. This efficiency gain is consistent with the critic-guided policy update. The learned value function favors action chunks that make reliable task progress. As a result, the policy reduces hesitations, retries, and unstable intermediate behaviors, rather than only improving eventual task completion.

Fig. 6 visualizes the value estimate during a successful and a failed Gongfu Tea episode. In the successful episode, the value tends to increase as the robot completes key sub-steps

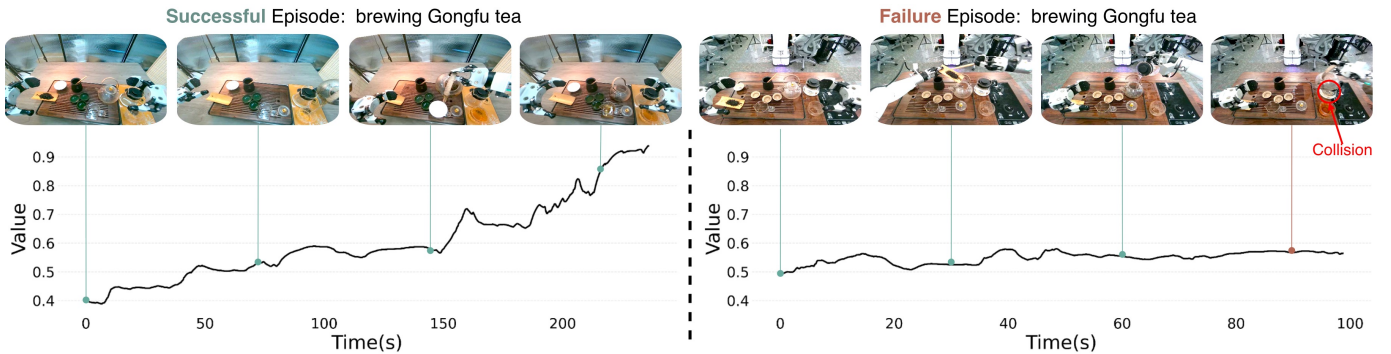


Fig. 6: **Visualizations of value learning.** We plot quantile values of the learned distributional value function V over time for representative Gongfu Tea episodes. The left trajectory succeeds and the right trajectory fails. The curves are qualitative diagnostics and are consistent with the learned value estimate tracking task-progress differences in these examples.

TABLE II: **Ablation of value learning design.** We report the average success rate on the short-horizon (grocery restocking tasks) and long-horizon tasks under offline and online settings.

Method	Short-Horizon		Long-Horizon	
	Offline	Online	Offline	Online
Expectile Regression	0.96	0.97	0.72	0.78
DIVL (Ours)	0.97 (+1.0%)	0.99 (+2.1%)	0.79 (+9.7%)	0.91 (+16.7%)

and approaches task completion, suggesting that the learned value can reflect progress despite sparse terminal rewards. In the failure episode, the value fluctuates locally but remains lower after the execution stops making progress toward the annotated task milestones. Additional visualizations of the predicted value distributions are provided in Appendix Fig. 9.

C. Ablation Study

1) *Value Learning Design:* We compare DIVL with scalar expectile value regression while keeping all other components fixed. DIVL outperforms the scalar baseline on all tasks, with larger gains on long-horizon tasks (9.7% in the offline stage and 16.7% in the online stage). In fleet deployment, the replay buffer contains diverse successful, failed, and intervention trajectories collected across tasks and scenes. By compressing heterogeneous outcomes into a single expected value, a scalar value function blurs rare but reproducible high-return behaviors. A distributional value instead retains the return distribution, preserving these high-return modes and providing a more informative signal for policy improvement. Complete per-task results are reported in Appendix Table V.

2) *Adaptive τ Strategy:* We further ablate the adaptive τ strategy used in DIVL during offline LWD training. We compare the adaptive schedule against a constant- τ baseline, where the constant value ($\tau = 0.52$) is set to the average τ observed from training statistics in the adaptive- τ run; all other components are kept identical. Table III shows that adaptive τ improves the average offline score from 0.84 to 0.88. Although the constant baseline is competitive on a few individual tasks, the adaptive schedule gives more consistent gains across all

tasks, especially on Restocking, Correction, and Cocktail. This indicates that conditioning τ on distributional entropy helps calibrate bootstrap optimism, making targets more conservative under high uncertainty and more optimistic when the value estimate is confident.

VI. CONCLUSION

We present Learning While Deploying (LWD), a large-scale real-world reinforcement learning framework for post-training generalist robot policies. LWD first initializes the policy from previously collected robot data, then continues improving it through online RL during deployment. The framework uses DIVL for value learning and QAM for policy extraction. Across eight real-world manipulation tasks spanning grocery restocking and long-horizon manipulation, LWD delivers the best overall performance, with the most pronounced improvements on long-horizon tasks.

These results suggest a practical path toward large-scale real-world deployment of continuously improving robot systems. With LWD, deployment is not only the setting in which the policy is evaluated, but also the mechanism through which the policy improves. Interaction data collected from the robot fleet is aggregated into a shared learning process, enabling a generalist policy to continue improving across tasks. This is critical for real-world robotic systems that must operate in heterogeneous tasks and environments.

Our method has several limitations. First, the current online learning pipeline updates with a straightforward real-time schedule. This design may not be optimal for larger-scale deployment or long-term continual improvement. More efficient and stable update strategies remain an important direction for future work. Second, our long-horizon experiments rely on a single short language instruction for each task. However, complex tasks require stronger vision-language reasoning for task decomposition, as well as finer-grained prompts for closed-loop execution and error recovery. Third, our current policy learning framework does not explicitly model execution safety. Incorporating safety-aware learning and control mechanisms will be important for reliable real-world deployment. Despite these limitations, this work represents a step toward large-scale

TABLE III: **Ablation of the adaptive τ strategy in offline LWD.** We compare the adaptive τ schedule with a constant τ baseline. For the constant baseline, τ is set to the empirical average value of the adaptive schedule from the adaptive- τ run ($\tau = 0.52$), while all other training components are kept unchanged.

Method	Grocery Restocking Tasks				Long-Horizon Tasks				Average
	Restocking	Correction	Freezer	Open-Cooler	Gongfu Tea	Fruit Juice	Cocktail	Shoobox	
LWD Offline, constant τ	0.85	0.88	0.94	0.95	0.70	0.76	0.70	0.90	0.84
LWD Offline, adaptive τ	1.00	1.00	0.92	0.95	0.72	0.74	0.83	0.86	0.88

real-world deployment, with the long-term goal of continuously scaling robot learning systems for robust execution in unstructured environments.

VII. ACKNOWLEDGMENTS

We thank Qiyang Li for helpful discussions.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [2] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [3] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna *et al.*, “Octo: An open-source generalist robot policy,” *arXiv preprint arXiv:2405.12213*, 2024.
- [4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [5] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [6] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *9th Annual Conference on Robot Learning*, 2025.
- [7] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, “Hg-dagger: Interactive imitation learning with human experts,” in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 8077–8083.
- [8] C. J. Watkins and P. Dayan, “Q-learning,” *Machine learning*, vol. 8, no. 3, pp. 279–292, 1992.
- [9] S. Fujimoto, H. Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International conference on machine learning*. PMLR, 2018, pp. 1587–1596.
- [10] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. M. O. Heess, T. Erez, Y. Tassa, D. Silver, and D. P. Wierstra, “Continuous control with deep reinforcement learning,” Sep. 15 2020, uS Patent 10,776,692.
- [11] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International conference on machine learning*. PMLR, 2018, pp. 1861–1870.
- [12] K. Lei, H. Li, D. Yu, Z. Wei, L. Guo, Z. Jiang, Z. Wang, S. Liang *et al.*, “RI-100: Performant robotic manipulation with real-world reinforcement learning,” *arXiv preprint arXiv:2510.14830*, 2025.
- [13] Y. Li, X. Ma, J. Xu, Y. Cui, Z. Cui, Z. Han, L. Huang, T. Kong *et al.*, “Gr-rl: Going dexterous and precise for long-horizon robotic manipulation,” *arXiv preprint arXiv:2512.01801*, 2025.
- [14] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, “Conrft: A reinforced fine-tuning method for vla models via consistency policy,” *arXiv preprint arXiv:2502.05450*, 2025.
- [15] A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia *et al.*, “ $\pi_{0.6}^*$: a vla that learns from experience,” *arXiv preprint arXiv:2511.14759*, 2025.
- [16] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn *et al.*, “Serl: A software suite for sample-efficient robotic reinforcement learning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 16 961–16 969.
- [17] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning,” *Science Robotics*, vol. 10, no. 105, p. eads5033, 2025.
- [18] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang, “Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning,” *arXiv preprint arXiv:2505.18719*, 2025.
- [19] S. Tan, K. Dou, Y. Zhao, and P. Krähenbühl, “Interactive post-training for vision-language-action models,” *arXiv preprint arXiv:2505.17016*, 2025.
- [20] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, Q. Zhang *et al.*, “ π rl: Online rl fine-tuning for flow-based vision-language-action models,” *arXiv preprint arXiv:2510.25889*, 2025.
- [21] J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan,

- D. Zhang *et al.*, “Flow-grpo: Training flow matching models via online rl,” *arXiv preprint arXiv:2505.05470*, 2025.
- [22] T. Zhang, C. Yu, S. Su, and Y. Wang, “Reinflow: Fine-tuning flow matching policy with online reinforcement learning,” *arXiv preprint arXiv:2505.22094*, 2025.
- [23] I. Kostrikov, A. Nair, and S. Levine, “Offline reinforcement learning with implicit q-learning,” *arXiv preprint arXiv:2110.06169*, 2021.
- [24] C. Domingo-Enrich, M. Drozdal, B. Karrer, and R. T. Chen, “Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control,” *arXiv preprint arXiv:2409.08861*, 2024.
- [25] Q. Li and S. Levine, “Q-learning with adjoint matching,” *arXiv preprint arXiv:2601.14234*, 2026.
- [26] M. Nakamoto, S. Zhai, A. Singh, M. Sobol Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine, “Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 62 244–62 269, 2023.
- [27] Z. Zhang, K. Zheng, Z. Chen, J. Jang, Y. Li, S. Han, C. Wang, M. Ding *et al.*, “Grape: Generalizing robot policy via preference alignment,” *arXiv preprint arXiv:2411.19309*, 2024.
- [28] H. Zang, M. Wei, S. Xu, Y. Wu, Z. Guo, Y. Wang, H. Lin, L. Shi *et al.*, “Rlinf-vla: A unified and efficient framework for vla+ rl training,” *arXiv preprint arXiv:2510.06710*, 2025.
- [29] J. Liu, F. Gao, B. Wei, X. Chen, Q. Liao, Y. Wu, C. Yu, and Y. Wang, “What can rl bring to vla generalization? an empirical study,” *arXiv preprint arXiv:2505.19789*, 2025.
- [30] C. Xu, Q. Li, J. Luo, and S. Levine, “Rldg: Robotic generalist policy distillation via reinforcement learning,” *arXiv preprint arXiv:2412.09858*, 2024.
- [31] C. Li, R. Zhang, J. Wong, C. Gokmen, S. Srivastava, R. Martin-Martin, C. Wang, G. Levine *et al.*, “Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation,” in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, pp. 80–93.
- [32] T. Mu, Z. Ling, F. Xiang, D. Yang, X. Li, S. Tao, Z. Huang, Z. Jia *et al.*, “Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations,” *arXiv preprint arXiv:2107.14483*, 2021.
- [33] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [34] Y. Mu, T. Chen, S. Peng, Z. Chen, Z. Gao, Y. Zou, L. Lin, Z. Xie *et al.*, “Robotwin: Dual-arm robot benchmark with generative digital twins (early version),” in *European Conference on Computer Vision*. Springer, 2024, pp. 264–273.
- [35] H. Zang, S. Yu, H. Lin, T. Zhou, Z. Huang, Z. Guo, X. Xu, J. Zhou *et al.*, “Rlinf-user: A unified and extensible system for real-world online policy learning in embodied ai,” *arXiv preprint arXiv:2602.07837*, 2026.
- [36] Z. Jiang, S. Zhou, Y. Jiang, Z. Huang, M. Wei, Y. Chen, T. Zhou, Z. Guo *et al.*, “Wovr: World models as reliable simulators for post-training vla policies with rl,” *arXiv preprint arXiv:2602.13977*, 2026.
- [37] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang, X. Zhu, Y. Zhang *et al.*, “Simplevla-rl: Scaling vla training via reinforcement learning,” *arXiv preprint arXiv:2509.09674*, 2025.
- [38] S. Park, Q. Li, and S. Levine, “Flow q-learning,” in *Forty-second International Conference on Machine Learning*, 2025.
- [39] L. Kun, Z. He, C. Lu, K. Hu, Y. Gao, and H. Xu, “Uni-o4: Unifying online and offline deep reinforcement learning with multi-step on-policy optimization,” in *The Twelfth International Conference on Learning Representations*.
- [40] S. Lee, Y. Seo, K. Lee, P. Abbeel, and J. Shin, “Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble,” in *Conference on Robot Learning*. PMLR, 2022, pp. 1702–1712.
- [41] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. Bellemare, “Reincarnating reinforcement learning: Reusing prior computation to accelerate progress,” *Advances in neural information processing systems*, vol. 35, pp. 28 955–28 971, 2022.
- [42] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient online reinforcement learning with offline data,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 1577–1594.
- [43] A. Nair, A. Gupta, M. Dalal, and S. Levine, “Awac: Accelerating online reinforcement learning with offline datasets,” *arXiv preprint arXiv:2006.09359*, 2020.
- [44] Y. Song, Y. Zhou, A. Sekhari, J. A. Bagnell, A. Krishnamurthy, and W. Sun, “Hybrid rl: Using both offline and online data can make rl efficient,” *arXiv preprint arXiv:2210.06718*, 2022.
- [45] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, “Steering your diffusion policy with latent space reinforcement learning,” *arXiv preprint arXiv:2506.15799*, 2025.
- [46] D. Kalashnikov, V. Vanhoucke, S. Levine, J. T. Springenberg, S. Bohez, K. Driessens, J. Schulman, M. Andrychowicz *et al.*, “Qt-opt: Scalable deep reinforcement learning for vision-based robotic manipulation,” in *Proceedings of the 2nd Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 87, 2018, pp. 651–673.
- [47] D. Kalashnikov, J. Varley, Y. Chebotar, B. Swanson, R. Jonschkowski, C. Finn, S. Levine, and K. Hausman, “Mt-opt: Continuous multi-task robotic reinforcement learning at scale,” *arXiv preprint arXiv:2104.08212*, 2021.
- [48] K.-H. Lee, T. Xiao, A. Li, P. Wohlhart, I. Fischer, and Y. Lu, “Pi-qt-opt: Predictive information improves multi-task robotic reinforcement learning at scale,” in

Conference on Robot Learning. PMLR, 2023, pp. 1696–1707.

- [49] M. Pan, S. Feng, Q. Zhang, X. Li, J. Song, C. Qu, Y. Wang, C. Li *et al.*, “Sop: A scalable online post-training system for vision-language-action models,” *arXiv preprint arXiv:2601.03044*, 2026.
- [50] K. Bousmalis, G. Vezzani, D. Rao, C. Devin, A. X. Lee, M. Bauzá, T. Davchev, Y. Zhou *et al.*, “Robocat: A self-improving generalist agent for robotic manipulation,” *arXiv preprint arXiv:2306.11706*, 2023.
- [51] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu *et al.*, “Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures,” in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1407–1416.
- [52] A. Herzog, K. Rao, K. Hausman, Y. Lu, P. Wohlhart, M. Yan, J. Lin, M. G. Arenas *et al.*, “Deep rl at scale: Sorting waste in office buildings with a fleet of mobile manipulators,” *arXiv preprint arXiv:2305.03270*, 2023.
- [53] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [54] M. G. Bellemare, W. Dabney, and R. Munos, “A distributional perspective on reinforcement learning,” in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 449–458.
- [55] A. Kumar, R. Agarwal, X. Geng, G. Tucker, and S. Levine, “Offline q-learning on diverse multi-task data both scales and generalizes,” *arXiv preprint arXiv:2211.15144*, 2022.
- [56] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, “Advantage-weighted regression: Simple and scalable off-policy reinforcement learning,” *arXiv preprint arXiv:1910.00177*, 2019.
- [57] S. Zhang, W. Zhang, and Q. Gu, “Energy-weighted flow matching for offline reinforcement learning,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [58] G. Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova *et al.*, “Gemma 3 technical report,” 2025.
- [59] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 11 975–11 986.
- [60] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
- [61] R. Ranftl, A. Bochkovskiy, and V. Koltun, “Vision transformers for dense prediction,” in *Proceedings of the*

IEEE/CVF international conference on computer vision, 2021, pp. 12 179–12 188.

- [62] J. Li, D. Li, S. Savarese, and S. Hoi, “Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in *International conference on machine learning*. PMLR, 2023, pp. 19 730–19 742.
- [63] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.

APPENDIX

A. Additional Method Details

1) *Discretization of Distributional Value Model*: We instantiate the distributional value model $V_\psi(s)$ with a fixed categorical support $\{V_i\}_{i=1}^K$ spanning $[v_{\min}, v_{\max}]$. In our real-robot experiments, we use $K = 201$ atoms over $[-0.1, 1.1]$. The value head predicts logits over this support,

$$p_\psi(i | s) = \text{softmax}(V_\psi(s))_i, \quad i \in \{1, \dots, K\}. \quad (20)$$

For each replay sample $(s, \mathbf{a})$, the scalar target $Q_{\bar{\phi}}(s, \mathbf{a})$ is clipped to $[v_{\min}, v_{\max}]$ and linearly projected onto the two neighboring atoms following the C51 projection [54]. This yields a target distribution $m(s, \mathbf{a})$ over atoms, and the distributional value model is trained by cross entropy:

$$\mathcal{L}_Z(\psi) = -\mathbb{E}_{(s, \mathbf{a}) \sim \mathcal{D}} \left[\sum_{i=1}^K m_i(s, \mathbf{a}) \log p_\psi(i | s) \right]. \quad (21)$$

The discrete CDF is

$$F_\psi(V_j | s) = \sum_{i \leq j} p_\psi(i | s), \quad (22)$$

and the quantile used in the DIVL TD target is obtained by selecting the first atom whose cumulative probability exceeds the desired level:

$$\text{Quant}_\tau(V_\psi(s)) = V_{\min\{j: F_\psi(V_j | s) \geq \tau\}}. \quad (23)$$

The normalized entropy used in the adaptive τ strategy is

$$\mathcal{H}(s) = -\frac{1}{\log K} \sum_{i=1}^K p_\psi(i | s) \log p_\psi(i | s) \in [0, 1]. \quad (24)$$

2) *Proof of the Distributional View of Asymmetric Value Estimation*: We provide the proof of Proposition 1 stated in Section IV-A. The goal is to show that, under idealized conditions, direct asymmetric optimization over dataset action-values and the two-step procedure of first fitting the state-conditioned distribution of dataset Q -values and then extracting the corresponding asymmetric statistic yield the same optimal scalar value.

Define the generalized asymmetric L_p loss

$$\rho_{\tau, p}(u) = |\tau - \mathbb{I}(u < 0)| \cdot |u|^p, \quad (25)$$

where $\tau \in (0, 1)$ is the asymmetry parameter. In standard IQL, the scalar value is obtained by directly minimizing

$$J_{\text{direct}}(v) = \mathbb{E}_{\mathbf{a} \sim \mathcal{D}(\cdot | s)} [\rho_{\tau, p}(Q(s, \mathbf{a}) - v)]. \quad (26)$$

The first-order optimality condition is

$$\frac{d}{dv} J_{\text{direct}}(v) = \int \mathcal{D}(\mathbf{a} | s) \cdot \frac{d}{dv} \rho_{\tau,p}(Q(s, \mathbf{a}) - v) d\mathbf{a} = 0. \quad (27)$$

Now consider DIVL in the idealized limit of infinitely fine discretization and sufficient model capacity. Let $p_\psi(z | s)$ denote the learned state-conditioned density over dataset Q -values. At optimum, the cross-entropy objective recovers the pushforward distribution induced by $\mathbf{a} \sim \mathcal{D}(\cdot | s)$ through the mapping $v = Q(s, \mathbf{a})$:

$$p_\psi(v | s) = P(v = Q(s, \mathbf{a}) | \mathbf{a} \sim \mathcal{D}(\cdot | s)). \quad (28)$$

Thus, for any integrable test function $f(z)$,

$$\mathbb{E}_{v \sim p_\psi(\cdot | s)}[f(z)] = \mathbb{E}_{\mathbf{a} \sim \mathcal{D}(\cdot | s)}[f(Q(s, \mathbf{a}))]. \quad (29)$$

The second step of DIVL extracts a scalar statistic by minimizing

$$J_{\text{dist}}(v) = \mathbb{E}_{u \sim p_\psi(\cdot | s)}[\rho_{\tau,p}(u - v)]. \quad (30)$$

Its first-order optimality condition is

$$\frac{d}{dv} J_{\text{dist}}(v) = \int p_\psi(u | s) \cdot \frac{d}{dv} \rho_{\tau,p}(u - v) du = 0. \quad (31)$$

Because $p_\psi(\cdot | s)$ is exactly the pushforward of $\mathbf{a} \sim \mathcal{D}(\cdot | s)$ under the random variable $u = Q(s, \mathbf{a})$, the above integral is identical to the direct objective's optimality condition after change of variables. Therefore, J_{direct} and J_{dist} admit the same minimizer v^* under the stated idealized assumptions.

This establishes that direct asymmetric optimization and the distribution-fit-then-extract procedure are equivalent in the limit. In particular, $p = 2$ recovers the expectile statistic used in standard IQL, while $p = 1$ recovers the quantile statistic used by DIVL.

3) *Analysis of Direct Backpropagation for Flow-Based Policy*: Consider a flow-based policy that generates an action $x = x_1$ by integrating the vector field $dx_t = f_\theta(x_t, t)$ from $t = 0$ to 1 starting from $x_0 \sim \mathcal{N}$. Writing $x_1 = x_1(x_0; \theta)$ for the terminal sample induced by the flow, the standard RL objective for reward fine-tuning is

$$J(\theta) = \mathbb{E}_{x_0 \sim \mathcal{N}}[R(x_1(x_0; \theta))], \quad (32)$$

and a vanilla policy gradient requires differentiating through the entire ODE trajectory:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x_0 \sim \mathcal{N}} \left[\nabla_x R(x_1) \cdot \int_0^1 \Phi(1, t) \frac{\partial f_\theta(x_t, t)}{\partial \theta} dt \right], \quad (33)$$

where $\Phi(1, t) = \frac{\partial x_1}{\partial x_t}$ is the sensitivity matrix along the flow. In practice, this formulation is computationally expensive and numerically fragile because it requires backpropagation through the full ODE solver [24]. Adjoint Matching (Section IV-B) avoids this issue by reformulating trajectory-level optimization as local regression targets along the flow path.

B. Implementation and Training Details

1) *Offline Data*: The offline buffer $\mathcal{B}_{\text{off}}$ consists of three types of data: *demonstration* data collected by human experts, *rollout* data produced by historical policies during prior evaluations, and *play* data in which a human operator explores failure modes and edge cases. Demonstrations are successful trajectories, rollouts contain both successes and failures, and play data is treated as unsuccessful exploratory data. Table IV summarizes the data composition in hours by task. Fig. 7 shows the aggregate source distribution, illustrating the relative contribution of demonstrations, historical rollouts, and play data.

2) *Training Hyperparameters*: The policy emits action chunks with horizon $H = 30$. The policy is optimized with AdamW [63] using a base learning rate of 2×10^{-5} and a cosine decay schedule. The value and critic networks are trained with Adam using a base learning rate of 5×10^{-4} , also with a cosine decay schedule.

For temporal-difference backups, we use $\gamma = 0.9999$. During offline training, we use $\tau_{\text{base}} = 0.6$ and uncertainty-sensitivity coefficient $\alpha = 0.3$ for DIVL. During online training, we use $\tau_{\text{base}} = 0.9$ and $\alpha = 0.3$. Target critic and value networks are updated with EMA rate 0.005, and the QAM policy-extraction temperature is $\lambda = 2$. The τ and entropy values during offline and online training are visualized in Fig. 8. Entropy decreases throughout offline-to-online training, indicating increasing confidence of value functions. Accordingly, the expectile parameter τ is increased, encouraging the policy to favor higher-value solutions.

For value learning, offline training uses 10-step chunk-level TD for long-horizon tasks and 1-step chunk-level TD for the grocery restocking tasks. Online training uses 1-step chunk-level TD for all tasks. During online training, each learner update samples mini-batches from $\mathcal{B}_{\text{off}} \cup \mathcal{B}_{\text{on}}$ with an approximately balanced ratio of 1:1.

3) *Checkpoint Initialization*: We first train an imitation-learning checkpoint by adapting the pretrained $\pi_{0.5}$ VLA policy on the demonstration data with behavior cloning. LWD (Offline) initializes its policy from this imitation-learning checkpoint, then trains the policy with the Adjoint Matching loss and trains the critic and distributional value model with DIVL. LWD (Online) initializes from the LWD (Offline) checkpoint, including both policy and value-learning modules, and continues training on mixed offline-online replay.

C. Additional Experimental Details

1) *Reference Policy and Baseline Implementations*: We obtain the reference policy by supervised fine-tuning [53] the pretrained $\pi_{0.5}$ VLA policy on 336.6 hours of demonstration data, as shown in Table IV. The model is trained with a flow-matching loss, where the interpolated noisy action $\mathbf{a}^w$ is defined in Eq. (7). The objective is to train conditional vector field $f_\theta(s, \mathbf{a}^w, w)$ to match the velocity $\mathbf{a}^1 - \mathbf{a}^0$ and minimizes:

$$\mathcal{L}_{\text{SFT}} = \mathbb{E} \left[\left\| f_\theta(s, \mathbf{a}^w, w) - (\mathbf{a}^1 - \mathbf{a}^0) \right\|_2^2 \right], \quad (34)$$

TABLE IV: **Offline data composition (hours)**. Demonstrations are expert-collected successful data; rollouts are generated by historical policies and contain both successes and failures; play data consists of human-guided explorations of failure modes.

Task	Demo	Rollout (Succ)	Rollout (Fail)	Play	Total
Restocking	14.5	10.7	1.7	7.5	34.4
Correction	12.7	10.8	1.7	9.7	34.9
Freezer	10.8	7.7	4.6	3.3	26.4
Open-Cooler	11.1	13.7	1.3	0.8	26.9
<i>Grocery Restocking Tasks (subtotal)</i>	<i>49.2</i>	<i>42.9</i>	<i>9.3</i>	<i>21.3</i>	<i>122.7</i>
Gongfu Tea	102.3	12.4	4.4	43.6	162.7
Fruit Juicing	100.5	17.4	14.7	47.1	179.8
Cocktail	47.3	4.4	7.4	28.0	87.1
Shoebox	37.3	11.7	3.3	48.0	100.3
<i>Long-Horizon Tasks (subtotal)</i>	<i>287.5</i>	<i>45.9</i>	<i>29.8</i>	<i>166.7</i>	<i>529.8</i>
All Tasks	336.6	88.8	39.2	187.9	652.5

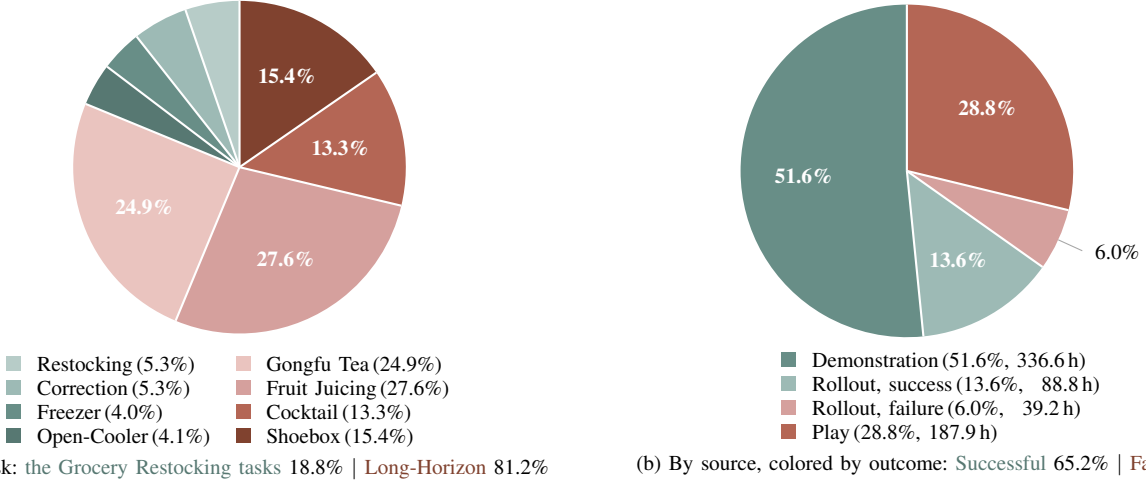


Fig. 7: **Offline data composition of the 652.5-hour buffer along two axes.** (a) Distribution across tasks: the grocery restocking tasks (green) and long-horizon tasks (red); long-horizon episodes dominate the buffer by volume due to their substantially longer duration. (b) Distribution across the three data sources—expert *demonstrations* (always successful), *rollouts* from historical policies (mixed successful and failure outcomes), and human-guided failure-mode *play* (always unsuccessful). Wedges are colored by trajectory outcome so that the overall success/failure split across the buffer is directly legible: roughly one-third of the buffer is failure data, which the behavior-cloning baselines cannot use but which provides an informative learning signal for LWD. Per-task hours are reported in Table IV.

And this reference policy is used for all the post-training methods.

lookahead advantage and binary improvement label as

$$A(s_t, \mathbf{a}_t) = \sum_{t'=t}^{t+H-1} r_{t'} + V(s_{t+H}) - V(s_t), \quad (35)$$

$$I_t = \mathbb{1}[A^{\pi_{\text{ref}}}(s_t, \mathbf{a}_t) > \epsilon \vee c_t = 1], \quad (36)$$

For the RECAP [15] baseline, we initialize from the reference policy and adapt RECAP to the eight-task generalist setting. We collect two rounds of autonomous rollouts: Round 1 uses the SFT checkpoint, and Round 2 uses the RECAP checkpoint obtained after training on Round 1. Each round contains approximately 60 robot-hours pooled across all eight tasks. Following RECAP, we train a value model to compute advantage labels over the combined dataset of demonstrations and both autonomous rollout rounds; the value model uses the same value-network architecture as LWD, described in Section IV-D, but only the value head. We compute the

where c_t indicates that the transition is a human intervention or correction, which is treated as positive following RECAP when present. We use $H = 30$ as our action horizon length and select a single global advantage threshold ϵ so that 30% of transitions in the combined training set satisfy the positive-advantage condition. This threshold is selected from training data only and is shared across all tasks to avoid task-specific tuning. After the second rollout round, we train RECAP for one epoch over the combined dataset and evaluate the resulting checkpoint.

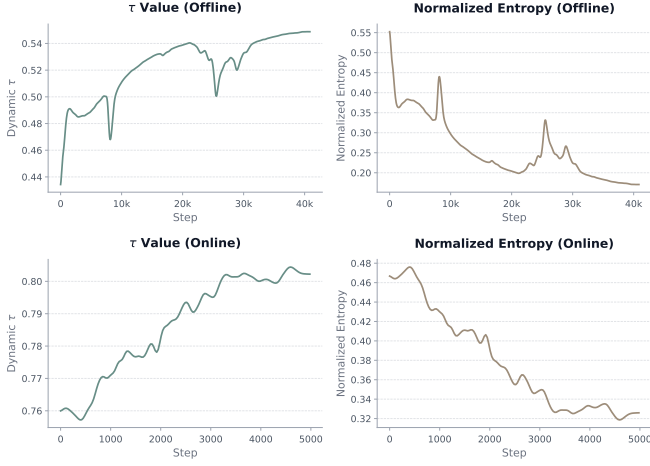


Fig. 8: **Dynamic τ and normalized entropy during offline-to-online training.** All curves are smoothed for readability. Entropy decreases throughout both stages, indicating increasing confidence in value estimation. Accordingly, τ is increased, leading to improved training performance.

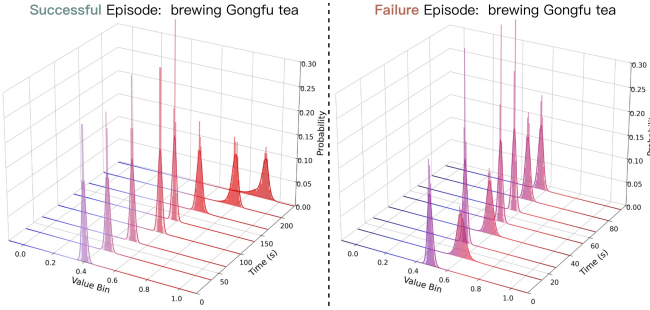


Fig. 9: **Predicted Value Distributions.** In the successful episode, the predicted distribution remains unimodal and its mode increases steadily from approximately 0.4 to 1.0. In contrast, the failure episode shows limited mode progression, rising only from approximately 0.5 to 0.6 before plateauing.

For the HG-Dagger [7] baseline, we initialize from the same reference policy checkpoint and run interactive imitation learning on the eight-task suite. During online execution, human operators provided intervention segments when corrections are needed. These intervention segments are aggregated with autonomous rollouts to form an online training buffer of approximately 60 robot-hours pooled across all eight real-world tasks. The online buffer, together with the offline demonstration data buffer, is used for the training of the HG-Dagger method. We train HG-Dagger from the reference policy checkpoint using the same batch size and training-time budget as the corresponding online post-training runs, and evaluate the resulting checkpoint.

For a fair comparison, the post-training baselines use the same policy optimizer and learning-rate schedule as LWD.

2) *Complete Value-Estimation Ablation Results:* Table V reports the complete per-task results for the value-estimation ablation summarized in Section V. The comparison isolates the

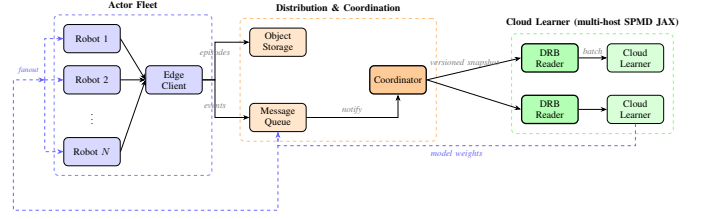


Fig. 10: **Distributed data infrastructure for LWD.** Robot actors upload episodes to object storage and publish event notifications to a message queue. A central *Coordinator* consumes notifications, fetches episode metadata, and commits versioned snapshots. The learner runs as a multi-host SPMD JAX program; on each node, the dataset (*DRB Reader*) holds a snapshot-bound view, spawns a prefetcher subprocess to download payloads from object storage, and feeds mini-batches to the local learner process. All DRB Readers synchronize on the same snapshot via a cross-host barrier. Updated model parameters produced by the collective are published back to all robot actors via the message-queue-backed publish-subscribe channel.

value-learning method by replacing DIVL with scalar expectile value regression while keeping the remaining training setup fixed.

3) *Complementary Qualitative Results of DIVL:* Fig. 9 visualizes the predicted value distributions for the same episodes shown in Fig. 6. In the successful episode, the predicted distribution remains unimodal, with its mode steadily increasing from approximately 0.4 to 1.0 as the task progresses. In contrast, the failure episode exhibits only marginal mode progression, increasing from approximately 0.5 to 0.6 before plateauing. These results indicate that the predicted value distribution provides a fine-grained signal to track policy progress and distinguish successful execution from failure cases.

D. Distributed Data Infrastructure

Fig. 10 illustrates LWD’s training data infrastructure, which links a fleet of robot actors to a multi-host learner via a versioned-snapshot data plane. On the actor side, each robot runs an edge client that accumulates per-frame observations into complete episodes and uploads them to distributed object storage at episode boundaries; episode metadata is persisted by a business service and event notifications are published to a message queue.

On the cloud side, a central *Coordinator* consumes event notifications from the message queue, fetches episode metadata from object storage, and commits monotonically increasing snapshot versions that define the training data view at each step. The learner runs as a multi-host SPMD JAX program, with one process per node driving all local accelerators. Each process instantiates a *Distributed Replay Buffer (DRB) Reader* as its dataset; before each training step, all DRB Readers synchronize on the same snapshot version via a cross-host barrier, ensuring the SPMD collective sees a globally consis-

TABLE V: **Ablation of value learning design (complete results)**. Complete results on grocery restocking tasks and long-horizon tasks. We compare continuous expectile regression and our distributional implicit value learning under offline and online settings. We report task success rate for each task and the average across all eight tasks. The best result per column is shown in bold.

Method		Grocery Restocking Tasks				Long-Horizon Tasks				Average
		Restocking	Correction	Freezer	Open-Cooler	Gongfu Tea	Fruit Juice	Cocktail	Shoebox	
Offline	Expectile Regression	0.85	1.00	1.00	1.00	0.68	0.74	0.71	0.76	0.84
	DIVL (ours)	1.00	1.00	0.92	0.95	0.72	0.74	0.83	0.86	0.88
Online	Expectile Regression	0.95	1.00	0.92	1.00	0.76	0.76	0.77	0.84	0.88
	DIVL (ours)	1.00	1.00	0.97	0.98	0.89	0.90	0.93	0.92	0.95

TABLE VI: **Operational Latency**. End-to-end latency measured on the same 8-hour, 16-actor online-RL run as the End-to-End Reliability subsection above. Absolute values are sensitive to network configuration and link contention and may vary across deployments.

Path	P50	P99
Episode produced → available to learner	41 s	148 s
Model published → received by actor	38 s	55 s

(i) *episode-to-learner*: the elapsed time from when an episode is produced on an actor to when it becomes available for the learner to sample; and (ii) *model-to-actor*: the elapsed time from when the learner publishes a new policy to when the actor has loaded it for the next rollout. Table VI reports both on the same 8-hour, 16-actor run as the End-to-End Reliability subsection above. Both latencies are dominated by object-storage I/O on the actor-to-cloud link — the episode payload in one direction, the policy artifact in the other — so absolute values are sensitive to link bandwidth and contention and may vary substantially across deployments.

tent dataset view despite asynchronous edge ingestion. Each DRB Reader spawns a prefetcher subprocess that downloads payloads from object storage in parallel; placing one prefetcher per node is sufficient to saturate the per-node read bandwidth available from the underlying distributed filesystem in our deployment.

Model parameters produced by the SPMD collective are published to a publish-subscribe channel that fans out to all robot actors, which reload the new policy at episode boundaries. Across the entire design, the Coordinator is the only orchestration singleton; both the actor fleet and the learner scale independently.

We characterize this infrastructure along two operational axes that are critical for online RL: whether every collected episode is reliably incorporated into training, and how quickly new data and updated policies traverse the actor-learner loop.

1) *End-to-End Reliability*: The system provides at-least-once end-to-end delivery for every episode produced on the actor side. (i) Object-storage uploads commit atomically (readers see either the fully-uploaded payload or no object) and are retried until persisted. (ii) Episode metadata is committed via a transactional insert in the business service, then announced to a durable message queue with delivery acknowledgment, so notifications survive coordinator restarts. (iii) Per-node prefetcher download tasks are requeued on failure with bounded retries; on snapshot commit, the snapshot data and the version pointer are updated atomically, so partial failures cannot leave a snapshot inconsistent. In our profiled 8-hour, 16-actor run of 1,604 episodes, every episode ingested in steady state completed the full end-to-end pipeline.

2) *Operational Latency*: We report the two end-to-end latencies that govern the tightness of the actor-learner loop: